SYSTEMVERILOG · HOMEWORK

UART TX

설계 · 검증 계획서

uart_tx · uart (FSM) + b_tick_gen · 9600 bps · 8-N-1 · 100 MHz

environment (heap)

generator ▶ driver ▶ DUT ▶ monitor ▶ scoreboard

tx 하강 → bit 중앙 샘플 → 10,416 clk 주기 수집 → expect_data 비교

gen2drv_mbox · mon2scb_mbox · gen_next_evt · mon_next_evt

정안성

2026. 11

설계

4-state FSM

IDLE · START · DATA · STOP, b_tick 16개 = 1 bit

타이밍

1 bit = 10,416 clk

104.16 µs · 1 frame 10 bit = 1.0416 ms

검증 구조

gen → drv → DUT → mon → scb

mailbox 2개 · event 2개 · virtual interface

판정

7개 항목 PASS / FAIL

bit 폭 · start/data/stop · state별 tx · busy 중 입력 무시 · IDLE

01 **설계** — 사양 · 블록도 · FSM / ASM

02 **타이밍** — 1 frame 타이밍 · 파형

03 **검증 환경** — 구조 · 1 transaction 흐름

04 **판정** — PASS / FAIL · 시나리오

1 설계

SystemVerilog로 UART TX를 FSM 구조로 구현하고 ASM chart로 동작 정리

`SV` `FSM` `ASM`

2 검증 plan

블록도 · 타이밍 · D/M event 흐름도로 검증 환경과 판정 기준 정의

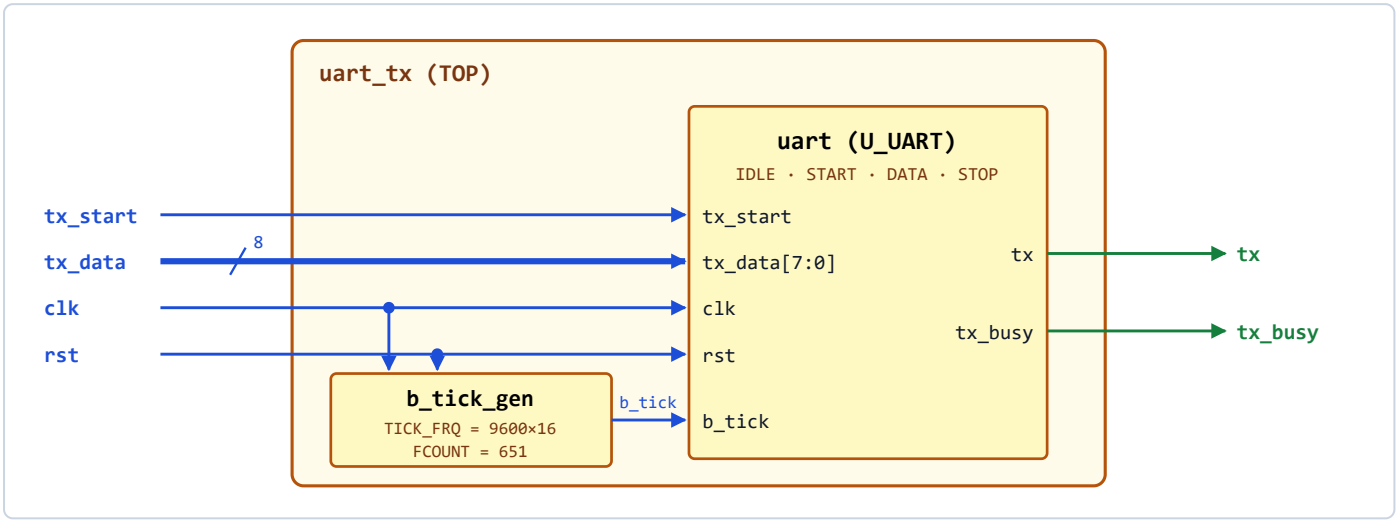
`B/D` `Timing` `D/M event 흐름도`

01

설계: b_tick_gen의 16× tick으로 FSM이 1 bit씩 송신

결론 · uart_tx = b_tick_gen (153.6 kHz tick) + uart FSM, tick 16개마다 1 bit 출력
· tx_start는 IDLE에서만 받아 tx_data를 data_reg에 latch → LSB부터 직렬 송신

블록도 uart_tx (TOP) 입력 = 파랑, 출력 = 초록



포트 module uart_tx

port	dir	width	설명
clk	in	1	100 MHz 시스템 클럭
rst	in	1	비동기 reset (high)
tx_start	in	1	전송 요청, IDLE에서만 수용
tx_data	in	8	송신 데이터, 수용 시 latch
tx	out	1	직렬 출력, idle = 1
tx_busy	out	1	프레임 전송 중 1

계산 baud tick

TICK_FRQ	**9600 × 16** = 153,600 Hz
FCOUNT	**10^8 / 153,600** = 651 clk
1 bit	**651 × 16** = 10,416 clk
time	**10,416 × 10 ns** = 104.16 µs
1 frame	**× 10 bit** = 104,160 clk

프레임 8-N-1 Start 1 · Data 8 (LSB first) · Parity 없음 · Stop 1

IDLE	START	D0	D1	D2	D3	D4	D5	D6	D7	STOP	IDLE
tx = 1	0	LSB							MSB	1	tx = 1

모듈 구성 요소

b_tick_gen
· r_counter 0 ~ 650 반복 카운트
· 650에서 b_tick 1 clk 펄스
· 153.6 kHz = baud × 16

uart · always_ff
· state · 카운터 · data_reg 레지스터
· rst 시 IDLE, tx = 1, busy = 0
· posedge clk마다 next 값 반영

uart · always_comb
· next state · 출력 결정
· 기본값 = 현재 값 유지 (latch 방지)
· b_tick_cnt 4 bit · bit_count 3 bit

01

FSM: 4-state, 모든 전이는 b_tick_cnt == 15에서

결론 · 모든 state 전이 조건 = b_tick & b_tick_cnt == 15 (16 tick = 1 bit)
· DATA에서 data_reg >> 1 · bit_cnt++, bit_cnt == 7이면 STOP → tx_busy 0 → IDLE

state 출력 · 동작 · 전이

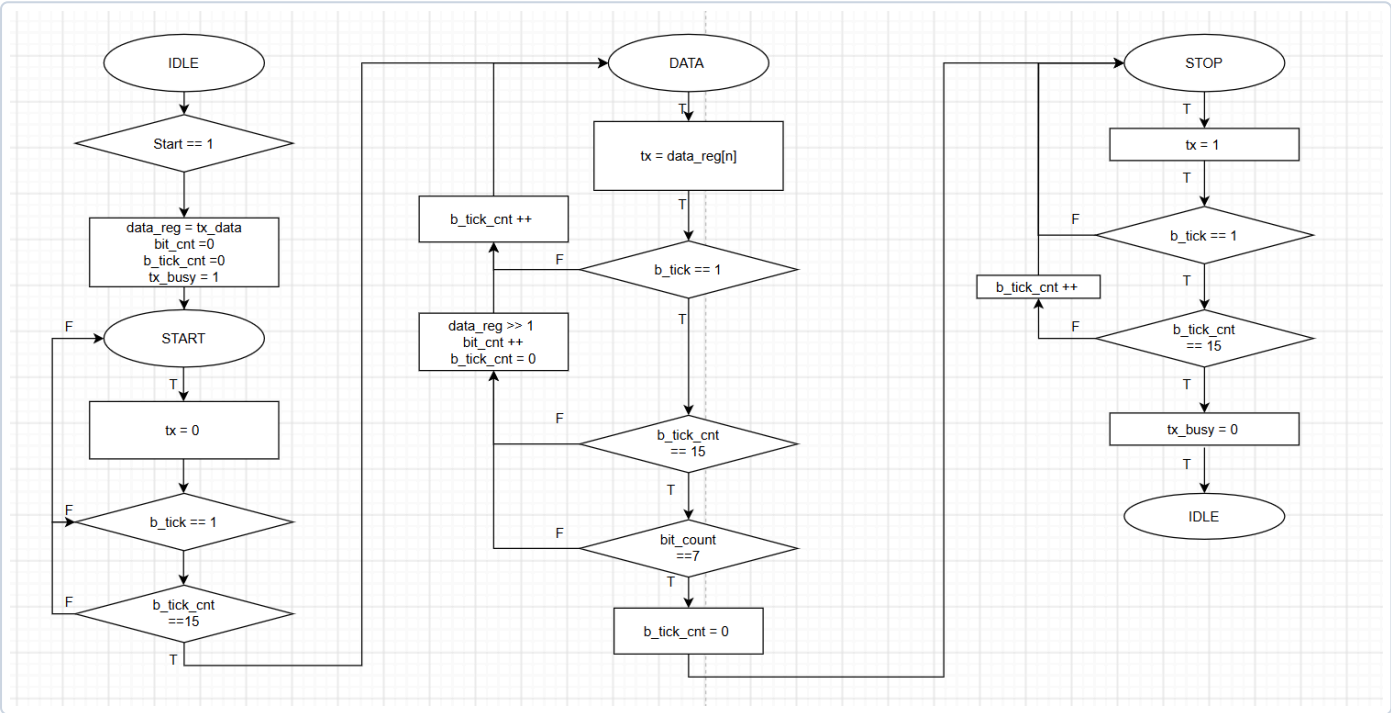
IDLE
tx = 1 · busy = 0
tx_start = 1이면 data_reg ← tx_data, bit_cnt · b_tick_cnt ← 0
→ **START** : tx_start

START
tx = 0 · busy = 1
b_tick마다 b_tick_cnt++
→ **DATA** : cnt == 15

DATA
tx = data_reg[0]
16 tick마다 data_reg >> 1, bit_cnt++
→ **STOP** : cnt == 15 & bit_cnt == 7

STOP
tx = 1 · busy = 1
stop bit 1 bit 유지
→ **IDLE** : cnt == 15

ASM UART TX ASM chart



설계 메모 RTL에서 검증할 포인트

1 tx_start는 IDLE에서만 검사
START·DATA·STOP에는 tx_start 분기가 없음 → busy 중 들어온 tx_start / tx_data 무시 (판정 #6)

2 tx · tx_busy는 레지스터 출력
START의 next 로직에서 tx_next = 0, tx_busy_next = 1 → tx_start 수용 후 다음 clk에 함께 변화

3 ASM과 RTL의 tx_busy 시점 차이
ASM은 IDLE 조건 박스에서 tx_busy = 1, RTL은 START에서 1 → 1 clk 차이, 파형으로 확인

4 LSB first
tx = data_reg[0] 출력 후 오른쪽 shift → D0부터 D7 순서, monitor도 같은 순서로 조립

레지스터 reset 값 판정 #7 (IDLE) 기준

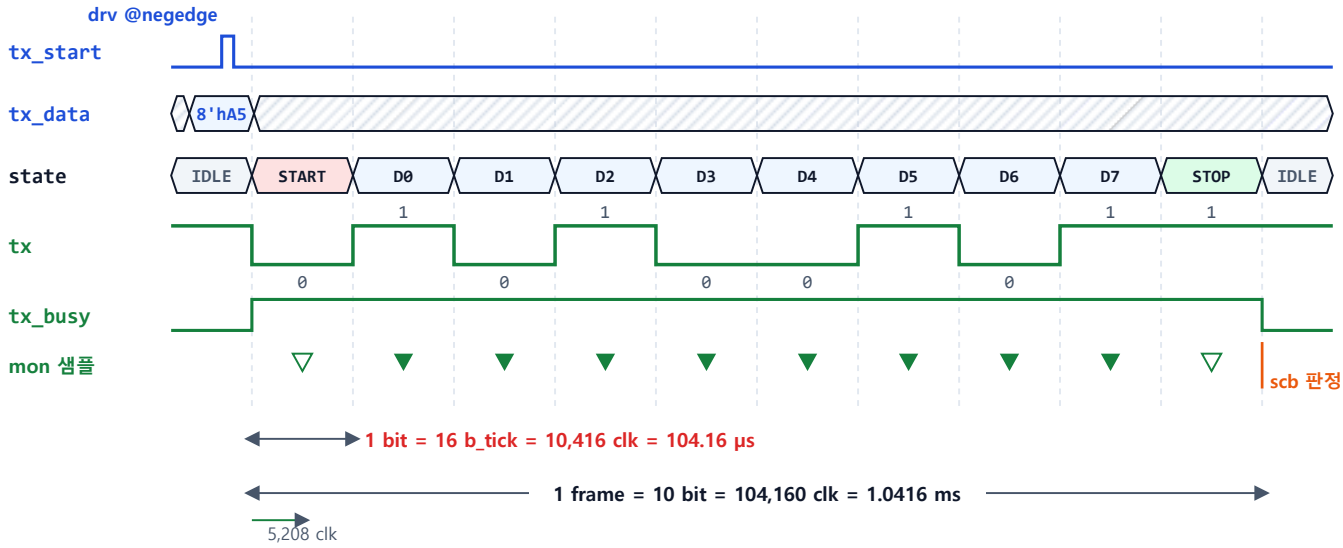
c_state	bit_count	b_tick_cnt	data_reg	tx_reg	tx_busy_reg
IDLE	3'b0	4'b0	8'h00	1	0

02

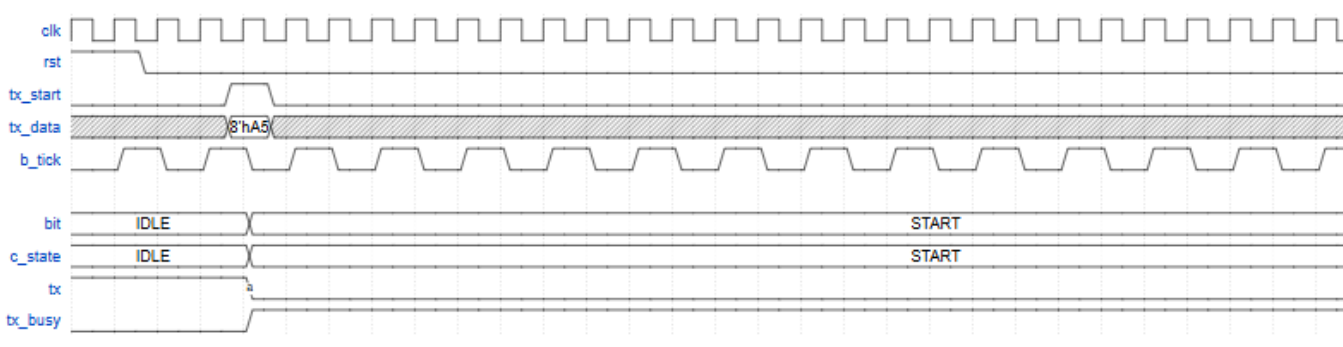
타이밍: 1 bit = 10,416 clk (104.16 µs), 1 frame = 10 bit

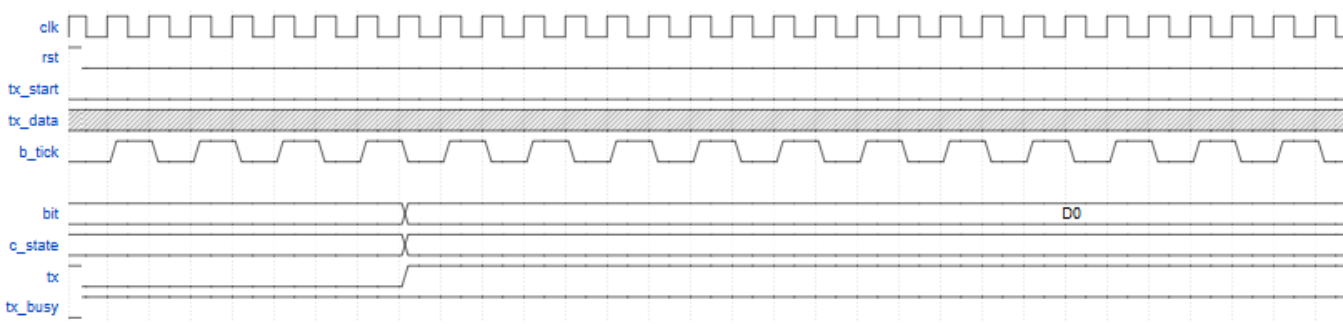
결론 · FCOUNT = 100 MHz / (9600 × 16) = 651 → 651 × 16 = 10,416 clk / bit
· monitor는 tx 하강 후 5,208 clk(bit 중앙)에서 첫 샘플, 이후 10,416 clk 주기로 D0 ~ D7 수집

timing `tx_data = 8'hA5` 1010_0101 → LSB first 출력 순서 D0 ~ D7 = 1 0 1 0 0 1 0 1



xsim 시뮬레이션 파형 Start → D0 구간





1 rst 해제 → tx_start
tx_start 1 clk 펄스와 함께 tx_data = 8'hA5 입력

2 START 진입
tx = 0, tx_busy = 1 동시에 변화, b_tick 16개 동안 유지

3 D0 진입
16번째 b_tick 후 tx = data_reg[0] = 1

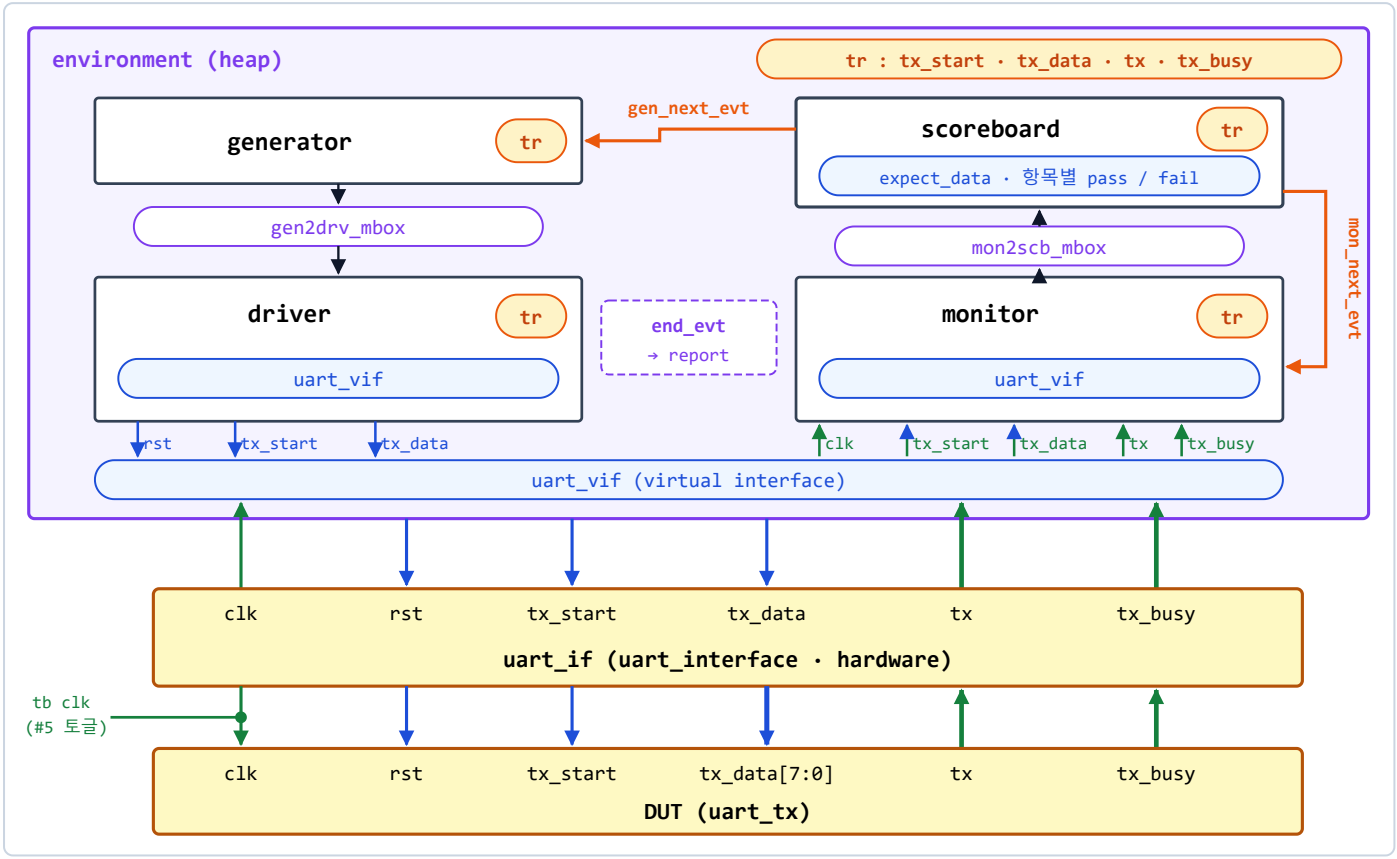
시점 Start 하강 기준 clk 카운트 Dn 중앙 = 5,208 + (n + 1) × 10,416

0	Start 하강 · 기준점	5,208	Start 중앙 · tx = 0	15,624	D0 중앙 샘플
88,536	D7 중앙 샘플	98,952	Stop 중앙 · tx = 1	≈104,160	frame 종료 → 판정

03

검증 환경: gen → drv → DUT → mon → scb 구조

결론 · generator의 tr을 driver가 DUT에 넣고, monitor가 복원한 8 bit를 scoreboard가 판정
· driver · monitor는 uart_if를 virtual interface로 공유 (driver 구동, monitor 관측)



generator
- tr randomize → gen2drv_mbox.put
- @(gen_next_evt) 대기 → 다음 tr
- N회 완료 → end_evt

driver
- get → @(negedge) 입력 구동
- rst · tx_start · tx_data
- negedge 구동으로 setup 여유

monitor
- posedge + 1 시점 샘플
- tx 하강 → bit 중앙 · 10,416 clk 주기로 8 bit 복원
- mon2scb_mbox.put

scoreboard
- START 진입 시 expect_data ← tr.tx_data
- 7개 항목 pass / fail 누적
- 판정 후 mon / gen_next_evt

environment
- mailbox · event 생성 및 연결
- preset → fork … join_none
- end_evt 수신 → report

transaction
- tx_start : dist {0 :/ 60, 1 :/ 40}
- tx_data : inside {[0:255]}
- tx · tx_busy : monitor 관측값

타이밍 규칙 race 없이 구동 · 샘플

1 negedge 구동
DUT가 posedge에서 읽기 전 반 주기 동안 입력이 안정

2 posedge + 1 샘플
non-blocking 갱신이 끝난 뒤 tx · tx_busy를 읽음

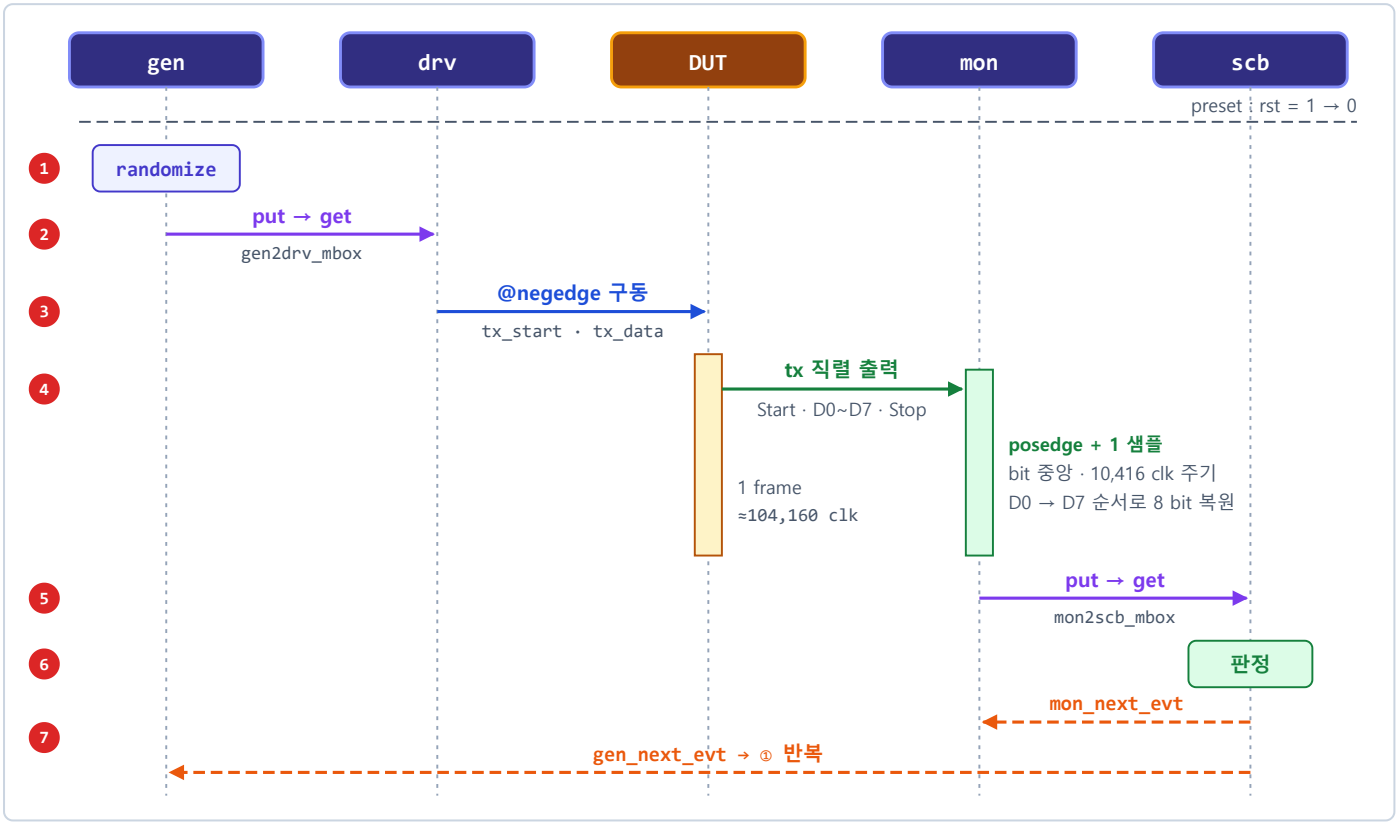
3 tb clk #5 토글
주기 10 ns = 100 MHz, DUT 분주 계산과 일치

03

1 transaction: put → get → 구동 → 샘플 → 판정

결론 · 판정이 끝난 뒤 mon_next_evt · gen_next_evt로 다음 tr 진행 → 한 번에 1 frame만 검증
· 기댓값은 scoreboard flag가 START 진입을 잡은 시점의 tr.tx_data → expect_data

D/M event 흐름도 파랑 = 구동, 초록 = 관측, 보라 = mailbox, 주황 = event



① gen	gen_next_evt 수신 후 tr randomize	⑤ mon→scb	mon2scb_mbox put → get
② gen→drv	gen2drv_mbox put → get	⑥ scb	expect_data와 비교, 7개 항목 pass / fail
③ drv→DUT	@negedge에 tx_start · tx_data 구동	⑦ scb→mon	mon_next_evt → 다음 frame 샘플 허용
④ DUT→mon	tx 하강 검출 → bit 중앙 샘플 → 8 bit 복원	⑦ scb→gen	gen_next_evt → ① 반복, N회 후 end_evt

기댓값 산출 scoreboard 내부 flag

1 START 진입 감지
flag로 DUT가 START state에 들어간 시점을 잡음

2 expect_data 저장
그 시점의 tr.tx_data를 logic [7:0] expect_data에 저장

3 busy 중 값은 무시
이후 들어온 tx_data는 기댓값을 바꾸지 않음 → 판정 #6과 연결

동기화 mailbox · event

이름	종류	보냄 → 받음	목적
gen2drv_mbox	mailbox	gen → drv	생성한 tr 전달
mon2scb_mbox	mailbox	mon → scb	복원한 8 bit · 관측값 전달
gen_next_evt	event	scb → gen	판정 완료 후 다음 tr 생성 허용
mon_next_evt	event	scb → mon	판정 완료 후 다음 frame 샘플 허용
end_evt	event	gen → env	N회 완료 → report 출력

04

판정: 7개 항목 PASS / FAIL, 4개 시나리오로 전부 커버

결론 · bit 폭 · start · data · stop · state별 tx · busy 중 입력 · IDLE을 tr마다 판정
· 시나리오 S1 ~ S4가 항목 #1 ~ #7을 모두 커버 → report에서 항목별 pass / fail 집계

scoreboard PASS / FAIL 항목

#	항목	확인 방법	PASS 조건
1	**bit 폭**	첫 tx 구동(Start 하강) 시점 이후 bit 경계 사이 clk 카운트	매 bit 10,416 clk (104.16 µs)
2	**start bit**	START 구간 tx 값	1 bit 동안 tx = 0
3	**data bit**	DATA 구간에서 bit 중앙마다 1 bit씩 저장 (주기 10,416 clk), D0 → D7 조립	수집값 == expect_data (각 bit 일치)
4	**stop bit**	STOP 구간 tx 값, 종료 후 tx_busy	tx = 1 유지 → tx_busy = 0
5	**state별 tx**	각 state(IDLE · START · DATA · STOP) bit 구간 안에서 tx 변화 유무	bit 경계에서만 변화, 구간 내 변화 없음
6	**busy 중 입력**	tx_busy = 1 동안 tx_start / tx_data 재전송	진행 중 frame 데이터 변화 없음
7	**IDLE**	reset 직후 · frame 사이 tx, tx_busy	tx = 1, tx_busy = 0

시나리오 자극

S1 reset	rst = 1 인가 후 해제 → 초기 출력 확인
S2 단일	tx_start 1 clk 펄스 + tx_data = 8'hA5
S3 random	tx_start 60 : 40, tx_data 0 ~ 255로 N회
S4 busy	전송 중 tx_start = 1, 다른 tx_data 인가

커버리지 시나리오 × 항목

	1	2	3	4	5	6	7
S1	○	○	○	○	○	○	●
S2	●	●	●	●	●	○	●
S3	●	●	●	●	●	●	●
S4	○	○	●	○	○	●	○

report 출력 형식 end_evt 수신 후 environment가 호출

```
+------------------------------------------------+
                UART TX  REPORT
+------------------------------------------------+
  total tr        : %0d        tx_start = 1 : %0d
+------------------------------------------------+
  #   item          pass      fail
  1   bit_width     %0d       %0d
  2   start_bit     %0d       %0d
  3   data_bits     %0d       %0d
  4   stop_bit      %0d       %0d
  5   tx_stable     %0d       %0d
  6   busy_ignore   %0d       %0d
  7   idle          %0d       %0d
+------------------------------------------------+
  FAIL : time · expect_data · 수집값 · 불일치 bit 위치
```

판정 기준 세부 규칙

1 bit 폭 오차 0 clk
FCOUNT가 정수 651이므로 bit 폭은 항상 정확히 10,416 clk

2 bit 중앙 샘플
경계에서 ±5,208 clk 여유 → 경계 근처 값 흔들림 영향 없음

3 FAIL 후 계속 진행
FAIL은 로그로 남기고 다음 tr 진행 → 끝까지 집계